

Sistemas Distribuídos

Vladimir Emiliano Rocha

Nome: Rafaela Barbosa Trindade. **RA:** 11201921826.

1. Vídeo

Link do vídeo: <https://youtu.be/NPqxm-tyib0>

2. RMI

Para realizar a comunicação via RMI do peer com o servidor, foram necessárias a criação de 3 classes (em **RMI**), que serão descritas nas subseções a seguir.

2.1. Peer RMI

O PeerRMI é a classe de objetos serializáveis (objetos que podem ser transferidos utilizando uma cadeia de bytes) destinada a conter as informações dos peers, por isso ela implementa a interface Serializable (linha 8).

Ela contém:

- os atributos que guardam as informações (linhas 10 e 11);
- um construtor que dá o valor inicial aos atributos ao instanciar o objeto (linhas 14-17);
- o método equals sobrescrito para ser capaz de comparar se dois objetos da classe são iguais com base em suas entidades, classes e valores dos atributos (linhas 20-33).

2.2. Servidor RMI

A ServerRMInterface é a interface que declara todos os métodos obrigatórios para um peer que utiliza a RMI. Ela estende a interface Remote (linha 11), por isso, todos os seus métodos podem ser invocados de uma máquina virtual não local (JVM).

A ServerRMImplements é a classe que implementa a interface ServerRMInterface (a implementação de seus métodos será vista na seção seguinte) e estende a classe UnicastRemoteObject (linha 13), tendo um construtor que herda do construtor da classe estendida (linhas 21-23), assim, os

métodos podem exportar um objeto remoto e obter uma stub que se comunica com o objeto usando a comunicação ponto-a-ponto.

Ela também contém uma estrutura de dados, do tipo ConcurrentHashMap, para guardar as informações dos peers (linha 15). A estrutura ConcurrentHashMap é uma estrutura que possibilita trabalharmos com threads (que permitem realizar processamentos paralelos) sem que haja inconsistências de dados entre elas.

3. Servidor

3.1. Inicialização

Na classe Servidor (em **Servidor**) é realizada a inicialização do servidor no método estático main, que pode lançar as exceções RemoteException e AlreadyBoundException (linha 17).

Dentro do método main, é aberto um scanner para receber os dados inseridos pelos usuários via teclado, assim é recebido o ip do servidor e a porta em que o registro será criado, após isso, o scanner é fechado (linhas 21-34). O registro que terá os stubs (referências) dos objetos remotos então é criado e capturado (linhas 37-40). Por fim, é criado o stub do servidor (utilizando a ServerRMInterface) e vinculado o registro com seu nome e o stub (linhas 43-45).

Todas as impressões no console do servidor servem para avisar o usuário sobre quando inserir o ip e a porta e para informá-lo quando o servidor estiver rodando (linhas 23, 26, 30 e 48).

3.2. JOIN

O método join é utilizado por um peer que acabou de ser inicializado e deseja se conectar ao servidor. Ele recebe como parâmetro as informações mínimas do peer e a lista de arquivos que possui, podendo lançar uma RemoteException (linha 26).

Se a lista passada tiver ao menos um nome de arquivo, uma lista vazia de peers é criada e o peer (PeerRMI) é adicionado a ela (linhas 33-36). Cada nome de arquivo da lista é percorrido e, através de um if e do método putIfAbsent, é verificado se o nome do arquivo já está na estrutura de dados utilizada ou não (linhas 39-45). O método putIfAbsent insere os valores na estrutura se a chave primária passada (nome do arquivo) estiver ausente, retornando nulo, ou retorna

o valor associado a chave, o que faz o método update ser chamado, sendo passado o nome do arquivo e o peer como parâmetros (linha 42).

Note que, utilizando a chave primária como o nome do arquivo e os valores como os peers que o possuem, um peer que não tenha nenhum arquivo não precisa ser adicionado a estrutura.

Também são impressas no console do servidor as informações do peer adicionado com seus arquivos (linhas 29 e 44) e, por fim, o método retorna “JOIN_OK” (linha 48).

3.3. SEARCH

O método search, que também pode lançar uma RemoteException, quando requisitado, recebe o nome do arquivo buscado e o peer (PeerRMI) requerente como parâmetros, que são impressos no console do servidor como aviso de requisição feita (linhas 52, 54 e 55).

Utilizando o método computeIfPresent, retorna uma lista nula (chave não presente, isto é, não há peers com o nome de arquivo) ou a lista de peers que possuem o arquivo (linha 58).

Nota: utilizar o método computeIfPresent para pegar um valor ao invés do get() evita retornar um valor desatualizado (problemas de consistência), uma vez que não se pode executar o método mais de uma vez ao mesmo tempo (mecanismo chamado de locking), não é possível ler e atualizar os valores ao mesmo tempo (já que o método update, explicado a seguir, utiliza o mesmo método).

3.4. UPDATE

O método update recebe como parâmetros o nome do arquivo e o peer (PeerRMI) para incluir o peer na lista de peers do arquivo na estrutura de dados do servidor. Também pode soltar uma RemoteException (linha 62).

Utilizando o método computeIfPresent, explicado na subseção anterior (ver seção 3.3), se o nome do arquivo já estiver na estrutura, então um laço passa por cada peer da lista de peers do arquivo, verificando se o peer requerente e o peer da lista são iguais (linhas 63-75), para isso, utiliza o método equals da PeerRMI (ver seção 2.1).

Se os peers forem iguais, isto é, o peer já está na lista de peers do arquivo, o computeIfPresent retorna a lista de peers sem modificações (linha 70). Se ao final do laço, a lista de peers não for retornada, ou seja, o peer não está na lista, o

peer então é adicionado a lista que é retornada pelo `computeIfPresent` (linhas 73 e 74). Por fim, o método retorna “UPDATE_OK” (linha 77).

Nota: como o update não realiza atualizações na estrutura de dados do servidor se o peer requerente já estiver na lista de peers do arquivo que possui, o usuário pode requisitar o join quantas vezes desejar, pois o peer não será replicado na estrutura. Isto pode ser útil caso o peer tenha um novo arquivo em sua pasta e seja necessário realizar o join novamente.

4. Peer

A classe Peer (em **Peer**) é a classe responsável por receber e enviar as requisições RMI ao servidor. Ela possui os seguintes atributos: soquete do servidor, caminho da pasta do peer e a própria interface do servidor RMI para a comunicação com o servidor (linhas 29-31). Além de ter dois construtores: um genérico (36-38), criado somente para dar ao método estático `main` acesso aos métodos privados da classe, e um específico que auxilia na passagem do soquete do servidor para as threads (44-46). Ela também implementa a interface `Runnable` para a criação de threads (linha 27).

4.1. Inicialização e Menu Interativo

No método main, é aberto um Scanner que lerá todas as entradas do usuário via teclado que a classe precisar (linha 238), é criado um objeto Peer para ter acesso a todos os métodos da classe (linha 241) e também é criado o objeto remoto PeerRMI que receberá o ip e a porta do peer (linha 244), através do método `getInfo` (ver seção 4.2).

Logo a seguir, é criada e iniciada a **thread** responsável por realizar o processo paralelo de iniciar o servidor do peer e esperar por uma conexão de um peer externo via TCP. Ao iniciar a thread, é criado o socket do servidor que ficará a escuta da requisição de conexão do peer externo através de um laço while (linhas 250-251). Assim que um peer externo se conecta ao servidor do peer, é iniciada a thread da classe, passando como parâmetro o objeto Peer com o soquete do servidor (linhas 253-257).

Enquanto a thread executa os processos do peer como servidor, é capturado o registro RMI do servidor RMI (linhas 268-269), são declaradas variáveis de validação que auxiliarão na checagem de acesso do usuário a certas funções da classe (linhas 272-274) e é iniciado o laço do menu interativo (linha 276).

No menu interativo, o usuário tem as opções de conectar o peer a rede, buscar por um arquivo na rede e baixar um arquivo da rede, além de poder sair a qualquer momento. Lembrando que o usuário primeiro precisa se conectar a rede para assim poder buscar pelo arquivo e baixá-lo.

Na opção join, é chamado o método peerJoin (ver seção 4.4) que retorna “JOIN_OK” a uma das variáveis de validação (linha 286). Na opção search, se o peer se conectou a rede, então é recebido o nome do arquivo a ser procurado via teclado e logo é chamado o método peerSearch, que retorna a lista de peers que possuem o arquivo (ver seção 4.5) (linhas 296-297).

Por fim, na opção download, se o peer está conectado a rede e há pelo menos um peer na lista de peers que possuem o arquivo procurado, então é recebido do usuário o ip e a porta do peer escolhido e essas informações são guardadas em um objeto remoto do tipo PeerRMI (linhas 313, 315 e 318). A lista de peers que contém o arquivo é percorrida e é verificado se de fato possui o peer que o usuário escolheu, caso tenha, o método download (ver seção 4.6) é chamado, passando o ip, a porta e o nome do arquivo a ser baixado (linha 330), logo depois é realizada a atualização da lista de peers do arquivo (linha 333).

Caso o peer escolhido pelo usuário não exista ou não contenha o arquivo a ser baixado, o usuário é informado (linha 344), além de também ser informado caso não tenha buscado um arquivo que exista (linha 349), não ter se conectado a rede (linha 354) ou caso insira uma opção inválida do menu (linha 364).

4.2. GETINFO

O método getInfo é responsável por receber todas as informações do peer do teclado do usuário, como ip, porta e caminho da pasta do peer (linhas 57, 60 e 63), retornando um objeto PeerRMI com o ip e a porta do peer (linha 65).

4.3. GETFILES

O método getFiles é responsável por pegar todos os arquivos de vídeo da pasta do peer. Todas as extensões de arquivos de vídeo aceitas ficam armazenadas em uma lista do tipo ArrayList (linha 124). Assim, o método cria uma lista somente com os arquivos que possuem tal extensão (linha 127), com o auxílio do método FilenameFilter da classe File.

O método FilenameFilter pega somente os arquivos para os quais o método accept retorna verdadeiro. O accept sobrescrito pega a extensão do arquivo, pegando a substring do último ponto no nome do arquivo (linha 131), e verifica se

a lista de extensões contém a extensão do arquivo (134), retornando verdadeiro ou falso.

Tendo a lista de arquivos, o método `getFiles` retorna a lista vazia ou com os arquivos de vídeo do peer.

4.4. PEERJOIN

O método `peerJoin` é responsável por fazer a requisição `join` do peer ao servidor RMI. Antes de fazer a requisição, o método sempre pega os arquivos do peer (linha 153), assim, mesmo se já tiver feito o `join` e o usuário inserir um novo arquivo na pasta, o usuário pode dar `join` novamente para atualizar sua lista de arquivos (infelizmente não foi possível criar um método para caso o usuário exclua um arquivo, portanto, fica como ideia para ser implementada no futuro).

Após obter a lista de arquivos, o método realiza a requisição `join` ao servidor RMI (linha 156), chamando o método `join` (seção 3.2), passando a lista de arquivos e as informações do peer no objeto `PeerRMI`, que retorna `"JOIN_OK"`. Retornando a string, então o método imprime no console do peer as informações do peer (linha 160) e, se a lista de arquivos não estiver vazia, imprime também os arquivos que o peer possui (linhas 161-163).

Por fim, retorna `"JOIN_OK"`, confirmando ao peer a sua conexão com o servidor (linha 168).

4.5. PEERSEARCH

O método `peerSearch` é responsável por fazer a requisição `search` do peer ao servidor RMI, chamando o método `search` (seção 3.3) e passando o nome do arquivo a ser buscado e as informações do peer no objeto `PeerRMI` (linha 181). Ao receber a lista de peers que possuem tal arquivo, se a lista não for nula, é printado no console do peer os peers que possuem o arquivo procurado (linhas 184-187). Por fim, é retornada a lista de peers que tem o arquivo (linha 192).

4.6. DOWNLOAD

O método `download` é responsável por realizar, via TCP, o download do arquivo procurado pelo usuário. Ele recebe o ip e a porta do peer servidor, para tentar realizar a conexão, criando um soquete (linha 205), além de também receber o nome do arquivo a ser baixado.

Após a conexão ser estabelecida, é criado um canal de transmissão de saída de dados para o envio do nome do arquivo (linhas 210-211), um canal de transmissão de entrada de dados para o recebimento do tamanho do arquivo

(linhas 214-215) e um canal de transmissão de saída de dados de arquivos para a transferência dos dados do arquivo recebido para um arquivo na pasta do peer (linha 218).

Então é criado um buffer de 10KB para receber os bytes do arquivo (221), isto é, os pacotes de dados recebidos terão no máximo 10KB. Assim, é criado um laço que receba e transfira esses pacotes de dados para o arquivo na pasta do peer, até que todo o arquivo tenha sido transferido, ou seja, o tamanho do arquivo é reduzido a zero, ou até que não haja mais nenhum pacote com dados a ser recebido, o tamanho do buffer seja -1 (linhas 223-226).

Por fim, tendo recebido todos os dados, o canal de transmissão de saída de dados de arquivos é fechado (linha 260), assim como a conexão ao fechar o soquete (linha 233).

4.7. Transferência de arquivos

A transferência de arquivos do peer servidor para o peer externo é realizada dentro de uma **thread**. Após o peer externo se conectar e dar início a essa thread, é criado um canal de transmissão de entrada de dados para receber o nome do arquivo (linhas 75-76) e um canal de transmissão de saída de dados para enviar o tamanho do arquivo (linha 79 e 91).

Então é pego o arquivo a ser enviado (linha 84) e, se esse arquivo existir, é criado um canal de transmissão de entrada de dados de arquivos para receber os dados deste arquivo (linha 88). Um buffer de 10KB é criado para receber os bytes desse arquivo (linha 94) e, enquanto os pacotes tiverem dados a serem enviados (o tamanho do buffer não for -1), os pacotes são enviados para o peer externo (linhas 96-100).

Por fim, o canal de transmissão de entrada de dados de arquivos é fechado (linha 104), assim como a conexão com o peer externo, ao fechar o soquete (linha 108).

Note o arquivo ser fragmentado e enviado em pequenos pacotes de 10KB torna possível a **transferência de arquivos grandes**.

5. Referências

- Ler informações do teclado
[Reading input from Keyboard in Java \(beginwithjava.com\)](http://beginwithjava.com)
- Trabalhando com listas do tipo ArrayList

[ArrayList in Java - javatpoint](#)

- Listando arquivos de determinadas extensões
[List multiple image, video, text files \(extensions\) in directory/folder \(FilenameFilter/java\)](#)
- Trabalhando com o ConcurrentHashMap
[A Guide to ConcurrentMap | Baeldung](#)
[Reading and Writing With a ConcurrentHashMap | Baeldung](#)
- Comparando dois objetos com o método equals
[Comparing Objects in Java | Baeldung](#)
- A interface RMI
<https://youtu.be/SnJ95ITISRQ>
https://youtu.be/knCXDo_NhGA
- Conexão via TCP
<https://www.youtube.com/watch?v=nysfXweTI7o>
- Transferindo arquivos grandes via sockets
[File Transfer via Java Sockets](#)